SALES AND PRODUCT MANAGEMENT SYSTEM

-BY ANGAD KADAM

PROJECT OBJECTIVES

TO CREATE A STRUCTURED DATABASE FOR MANAGING SALES DATA.

TO ENSURE DATA INTEGRITY AND PREVENT DUPLICATION.

TO FACILITATE EASY ACCESS TO SALES AND PRODUCT INFORMATION FOR ANALYSIS.

DATABASE STRUCTURE

- Description of the four main tables:
  - **Geo:** Contains geographic information.
  - **People:** Stores salesperson details.
  - **Products:** Lists product information.
  - **Sales:** Records sales transactions.
- Mention of relationships and foreign keys between tables.

```
CREATE TABLE `sales` (
  `SPID` varchar(6),
  `GeoID` varchar(4),
  `PID` varchar(6),
  `SaleDate` datetime DEFAULT NULL,
  `Amount` int DEFAULT NULL,
  `Customers` int DEFAULT NULL,
  `Boxes` int DEFAULT NULL,
  foreign key(GeoID) references geo(GeoID),
  foreign key(SPID) references people(SPID),
  foreign key(PID) references products(PID)
```

HOW CAN WE MODIFY THE REGION COLUMN IN THE GEO TABLE TO ENSURE IT CANNOT CONTAIN NULL VALUES?
EXPLANATION: MODIFIES THE REGION COLUMN IN THE GEO TABLE TO DISALLOW NULL VALUES, ENSURING ALL RECORDS HAVE VALID DATA.

Input

```
alter table geo modify column Region text null;


describe geo;
```

Output

	Field	Type	Null	Key	Default	Extra
▶	GeoID	varchar(4)	NO	PRI	NULL	
	Region	text	NO		NULL	
	Country	text	YES		NULL	
	last_updated	datetime	YES		NULL	

Changing column position:
Explanation: This query changes the position of the columN ‘REGION’ TO BE IMMEDIATELY AFTER ‘GEOID’ COLUMN IN THE ‘GEO’ TABLE

Input

```
alter table geo modify column Region text after GeoID;
```

Output

GeoID	Region	Geo
G1	APAC	India
G2	Americas	USA
G3	Americas	Canada
G4	APAC	New Zealand
G5	APAC	Australia
G6	Europe	UK

TO SUMMARIZE TOTAL SALES AND BOXES SOLD BY PRODUCT CATEGORY:

Expalnation: CALCULATES TOTAL SALES AND BOXES SOLD BY PRODUCT CATEGORY.

Input

```
select  p.Category,
    SUM(s.Amount) AS Total_Sales_Amount,
    SUM(s.Boxes) AS Total_Boxes_Sold
from sales s
join products p on s.PID = p.PID
GROUP BY p.Category;
```

Output

	Category	Total_Sales_Amount	Total_Boxes_Sold
▶	Bars	471835	27135
	Bites	314440	17893
	Other	134498	6082

WHICH SALES RECORDS BELONG TO SPECIFIC SALESPERSONS (SPIDS) .

Explanation: RETRIEVES SALES RECORDS FOR SPECIFIED SALESPERSONS USING THE IN OPERATOR.

Input

```sql
SELECT *
FROM sales
WHERE SPID IN ('SP01', 'SP02', 'SP07', 'SP08');
```

Output

SPID	GeoID	PID	SaleDate	Amount	Customers	Boxes
SP01	G4	P04	2021-01-01 00:00:00	8414	276	495
SP01	G4	P15	2021-01-01 00:00:00	259	32	22
SP01	G5	P01	2021-01-11 00:00:00	7238	67	315
SP01	G3	P19	2021-01-12 00:00:00	490	188	35
SP01	G2	P14	2021-01-18 00:00:00	2331	38	167
SP02	G3	P14	2021-01-01 00:00:00	532	317	54
SP02	G4	P13	2021-01-01 00:00:00	2506	99	148
SP02	G1	P05	2021-01-08 00:00:00	2814	69	149
SP02	G3	P15	2021-01-12 00:00:00	2653	147	332
SP02	G2	P12	2021-01-14 00:00:00	14532	142	969
SP02	G4	P15	2021-01-14 00:00:00	4011	260	335
SP02	G4	P05	2021-01-15 00:00:00	3430	120	202
SP07	G4	P11	2021-01-05 00:00:00	7399	420	275
SP07	G4	P04	2021-01-05 00:00:00	4865	168	271
SP07	G2	P15	2021-01-06 00:00:00	1470	57	184
SP07	G4	P13	2021-01-13 00:00:00	2121	130	89
SP07	G6	P10	2021-01-19 00:00:00	8218	205	294

WHICH PRODUCT CATEGORIES HAVE NAMES STARTING WITH ‘B’ ?

EXPLANATION: RETRIEVES PRODUCTS WHOSE NAMES START WITH 'A' USING THE LIKE OPERATOR.

Input

```
select PID, Product, Category
from Products
where Category like 'B%';
```

Output

	PID	Product	Category
▶	P01	Milk Bars	Bars
	P02	50% Dark Bites	Bites
	P03	Almond Choco	Bars
	P04	Raspberry Choco	Bars
	P05	Mint Chip Choco	Bars
	P06	Eclairs	Bites
	P08	99% Dark & Pure	Bars
	P09	Orange Choco	Bars
	P10	Spicy Special Slims	Bites
	P11	After Nines	Bites
	P12	Fruit & Nut Bars	Bars
	P13	85% Dark Bars	Bars

WHAT ARE THE TOP 10 SALES RECORDS BY AMOUNT?

Explanation: RETRIEVES THE TOP 10 SALES RECORDS ORDERED BY AMOUNT USING THE LIMIT CLAUSE.

Input

```
select SPID, GeoID, PID, SaleDate,
       Amount, Customers, Boxes
from  sales
order by Amount desc limit 10;
```

Output

SPID	GeoID	PID	SaleDate	Amount	Customers	Boxes
SP18	G2	P21	2021-01-04 00:00:00	19229	64	1013
SP24	G2	P11	2021-01-14 00:00:00	18704	78	585
SP23	G1	P16	2021-01-05 00:00:00	17248	163	664
SP13	G2	P20	2021-01-19 00:00:00	16380	203	1092
SP05	G6	P20	2021-01-14 00:00:00	15785	209	1128
SP10	G1	P06	2021-01-01 00:00:00	15596	32	975
SP08	G1	P05	2021-01-19 00:00:00	15407	103	771
SP20	G5	P12	2021-01-14 00:00:00	14574	86	810
SP02	G2	P12	2021-01-14 00:00:00	14532	142	969
SP25	G6	P05	2021-01-01 00:00:00	14273	335	752

HOW CAN I LIST ALL SALES RECORDS ORDERED BY SALE DATE?

Explanation: LISTS ALL SALES RECORDS ORDERED BY SALE DATE IN ASCENDING ORDER.

Input

```
select SPID, GeoID, PID, SaleDate,
       Amount, Customers, Boxes
from sales
order by SaleDate desc;
```

Output

SPID	GeoID	PID	SaleDate	Amount	Customers	Boxes
SP19	G4	P21	2021-01-21 00:00:00	7805	86	488
SP09	G6	P11	2021-01-21 00:00:00	3087	14	124
SP13	G4	P21	2021-01-21 00:00:00	9030	85	452
SP04	G2	P20	2021-01-21 00:00:00	7805	6	411
SP18	G1	P11	2021-01-21 00:00:00	6979	38	233
SP19	G1	P07	2021-01-21 00:00:00	11284	39	513
SP17	G5	P01	2021-01-21 00:00:00	2408	106	90
SP03	G6	P02	2021-01-21 00:00:00	5299	86	332
SP10	G6	P16	2021-01-21 00:00:00	6293	158	234
SP08	G5	P03	2021-01-21 00:00:00	280	408	17
SP04	G4	P22	2021-01-20 00:00:00	1155	34	45
SP10	G6	P02	2021-01-20 00:00:00	784	247	49
SP25	G4	P05	2021-01-20 00:00:00	5369	16	384
SP10	G3	P06	2021-01-20 00:00:00	343	338	18
SP10	G6	P19	2021-01-20 00:00:00	7007	38	584
SP15	G1	P15	2021-01-20 00:00:00	126	127	13
SP16	G1	P08	2021-01-20 00:00:00	3073	57	134
SP11	G3	P16	2021-01-19 00:00:00	896	346	32
SP20	G1	P18	2021-01-19 00:00:00	4214	314	169
SP20	G3	P09	2021-01-19 00:00:00	2121	90	177

Calculate the total count of salespersons in each Team from the people table.
Explanation: This query calculates total number of salespersons in each team from the 'people' table

Input

```
select PID, COUNT(*) as TotalSales
from sales
group by PID;
```

Output

	PID	TotalSales
▶	P01	11
	P02	7
	P03	5
	P04	13
	P05	8
	P06	4
	P07	7
	P08	8
	P09	9
	P10	9
	P11	10
	P12	7
	P13	12
	P14	7
	P15	7

WHICH PRODUCTS HAVE TOTAL SALES GREATER THAN $1,000?

Explanation: FILTERS PRODUCTS WITH TOTAL SALES GREATER THAN $1,000 USING GROUP BY AND HAVING CLAUSES.

Input

```
select PID, sum(Amount) as TotalSales
FROM sales group by PID
having sum(Amount) > 1000;
```

Output

	PID	TotalSales
▶	P01	50694
	P02	40446
	P03	13293
	P04	42672
	P05	54131
	P06	17311
	P07	34356
	P08	39025
	P09	22302
	P10	63966
	P11	65821
	P12	60284
	P13	55034
	P14	30793
	P15	11396
	P16	32571
	P17	53410
	P18	36778
	P19	37856
	P20	69594
	P21	62188
	P22	26852

WHICH SALESPERSONS HAVE TOTAL SALES GREATER THAN THE TOTAL SALES OF A SPECIFIC PRODUCT?

Explanation: IDENTIFIES SALESPERSONS WHOSE TOTAL SALES EXCEED THOSE OF A SPECIFIED PRODUCT USING A SUB-QUERY FOR COMPARISON.

Input

```
select SPID, sum(Amount) as TotalSales
from sales group by SPID
having
sum(Amount) > (select sum(Amount) from sales where PID = 'P01');
```

Output

SPID	TotalSales
SP08	63749
SP10	54208
SP11	68509
SP18	62174
SP19	53459

WHAT ARE THE SALES RECORDS FOR THE SALESPERSON "BARR FAUGHNY" IN THE "BARS" CATEGORY, ORDERED BY SALES AMOUNT?

Explanation: RETRIEVES SALES RECORDS FOR "BARR FAUGHNY" IN THE "BARS" CATEGORY, ORDERED BY SALES AMOUNT IN DESCENDING ORDER.

Input

```
select p.Salesperson, p.Team, p.SPID, s.SPID, s.Amount,
       s.Customers, pr.Category, pr.Product, pr.PID
from sales s
join people p on s.SPID = p.SPID
join products pr on s.PID = pr.PID
where pr.category = "BARS" and p.Salesperson = "Barr Faughny"
order by s.Amount desc;
```

Output

Salesperson	Team	SPID	SPID	Amount	Customers	Category	Product	PID
Barr Faughny	Yummies	SP01	SP01	8414	276	Bars	Raspberry Choco	P04
Barr Faughny	Yummies	SP01	SP01	7238	67	Bars	Milk Bars	P01
Barr Faughny	Yummies	SP01	SP01	259	32	Bars	Baker's Choco Chips	P15

HOW CAN WE INSERT A NEW SALESPERSON INTO THE "PEOPLE" TABLE USING A STORED PROCEDURE?

Explanation: CREATES A STORED PROCEDURE TO INSERT A NEW SALESPERSON INTO THE "PEOPLE" TABLE, ENHANCING DATA MANAGEMENT EFFICIENCY.

Input

```
delimiter $$
create procedure insert_new_salesper(
    in p_Salesperson text,
    in p_SPID varchar(6),
    in p_Team text,
    in p_Location text
)
begin
    insert into people(Salesperson, SPID, Team, Location)
    values(p_Salesperson, p_SPID, p_Team, p_Location);
end$$
delimiter ;

CALL insert_new_salesper('Angad', 'SP34', 'Mango', 'Ratnagiri');
```

Output

	Salesperson	SPID	Team	Location
	Mallorie Waber	SP21		Seattle
	Jehu Rudeforth	SP22		Seattle
	Van Tuxwell	SP23	Yummies	Seattle
	Roddy Speechley	SP24	Delish	Seattle
	Camilla Castle	SP25	Delish	Seattle
	Janene Hairsine	SP26	Delish	Paris
	Niall Selesnick	SP27	Jucies	Paris
	Ebonee Roxburgh	SP28		Paris
	Zach Polon	SP29	Yummies	Paris
	Orton Livick	SP30	Yummies	Paris
	Gray Seamon	SP31	Delish	Paris
	Benny Karolovsky	SP32	Jucies	Paris
	Dyna Doucette	SP33	Jucies	Paris
	Angad	SP34	Mango	Ratnagiri

HOW CAN WE CALCULATE THE AVERAGE SALES AMOUNT FOR A SPECIFIC REGION USING A FUNCTION?
EXPLANATION: DEFINES A FUNCTION TO CALCULATE AVERAGE SALES FOR A GIVEN REGION, IMPROVING DATA ANALYSIS EFFICIENCY.

Input

```
delimiter $$
create function avg_sales_by_region(
e_Region text)
returns decimal(10,2)
deterministic
begin
    declare averageAmount decimal(10,2);
    select avg(s.Amount) into averageAmount
    from sales s join geo g
    on s.GeoID = g.GeoID
    where g.Region = e_Region;
    return averageAmount;
end$$
delimiter ;
select avg_sales_by_region("Americas");
```

Output

	avg_sales_by_region("Americas")
▶	6045.18

HOW CAN WE CREATE A TRIGGER TO PREVENT THE INSERTION OF DUPLICATE SALESPERSON IDS IN THE PEOPLE TABLE?

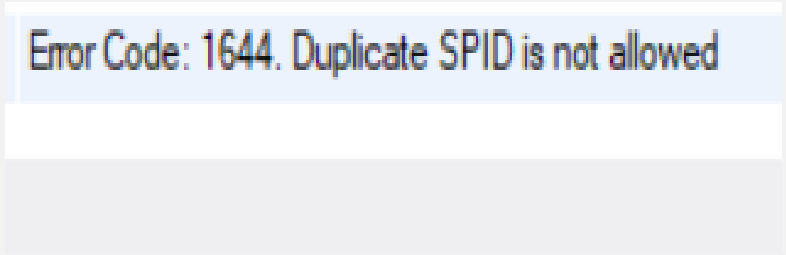
Explanation: CREATES A TRIGGER TO PREVENT DUPLICATE SALESPERSON IDS IN THE PEOPLE TABLE, ENSURING DATA INTEGRITY DURING INSERTIONS.

Input

```
delimiter $$
create trigger prevent_duplicate_spid
before insert on people
for each row
begin
    declare existing_count int;
    select count(*) into existing_count
    from people
    where SPID = new.SPID;

    if existing_count > 0 then
        signal sqlstate '45000'
        set message_text = 'Duplicate SPID is not allowed';
    end if;
end$$
DELIMITER ;
insert into people values("Angad", "SP01", "Mango", "Ratnagiri");
```

Output



HOW CAN WE CREATE A VIEW TO DISPLAY COMPREHENSIVE PRODUCT INFORMATION ALONG WITH SALES AND GEOGRAPHIC DETAILS?

EXPLANATION: CREATES A VIEW TO DISPLAY DETAILED PRODUCT AND SALES INFORMATION, STREAMLINING DATA RETRIEVAL FOR ANALYSIS.

Input

```
create view show_product_info as
select g.Geo, p.Salesperson,
       pr.Product, pr.Category,
       pr.Size, s.SaleDate,
       s.Amount, s.Boxes
from
geo g join sales s on s.GeoID = g.GeoID join
people p on p.SPID = s.SPID join
products pr on pr.PID = s.PID;

select * from show_product_info;
```

Output

Geo	Salesperson	Product	Category	Size	SaleDate	Amount	Boxes
India	Rafaelita Blaksland	85% Dark Bars	Bars	SMALL	2021-01-01 00:00:00	2184	122
India	Brien Boise	Eclairs	Bites	LARGE	2021-01-01 00:00:00	15596	975
India	Curtice Advani	After Nines	Bites	LARGE	2021-01-01 00:00:00	8561	330
India	Van Tuxwell	Organic Choco Syrup	Other	SMALL	2021-01-05 00:00:00	17248	664
India	Jan Morforth	Orange Choco	Bars	LARGE	2021-01-05 00:00:00	3059	279
India	Wilone O'Kielt	Smooth Sliky Salty	Bars	SMALL	2021-01-06 00:00:00	644	34
India	Ches Bonnell	After Nines	Bites	LARGE	2021-01-06 00:00:00	4935	171
India	Madelene Upcott	Manuka Honey Choco	Other	SMALL	2021-01-07 00:00:00	11228	388
India	Dennison Crosswaite	Mint Chip Choco	Bars	LARGE	2021-01-08 00:00:00	2814	149

CONCLUSION

In this project, I utilized SQL to manage and analyze sales data through key features such as joins for comprehensive reporting, stored procedures for inserting new salespersons, and functions to calculate average sales by region. Additionally, I implemented triggers to ensure data integrity by preventing duplicate entries. This approach demonstrated my ability to leverage database functionalities to drive informed business decisions and enhance operational efficiency.

THANK YOU